Fall Detection Project - Master SSE - Hanze

Ali Shirazi, Amir Masoud Jafari
a.shirazi@st.hanze.nl, a.jafari@st.hanze.nl

***Abstract*—This document provides instructions on how to prepare a report about your project. This guideline includes basic information that is expected in a report, along with a suggested structure for how to organize it. Examples of how to include figures and tables in LaTeX and a general style guide are also given. The usage of AI tools is also discussed. The text ends with a conclusion and a list of cited references.**

I. Introduction

Falls are a major health hazard for the elderly: roughly one-third of people over 65 experience a fall each year, often resulting in fractures or long periods of incapacitation [1]. Prompt fall detection and alerting can greatly reduce the severity of outcomes by preventing "long lie" situations [1], [2]. Wearable devices (e.g. smartphones, pendants, smartwatches) have been widely studied for this purpose, typically using accelerometers (and often gyroscopes) to detect the high accelerations or orientations associated with falls [3], [4]. In this work we propose an ear-worn device (earpiece) with an integrated 3-axis accelerometer (LIS2DW12) to automatically detect falls and notify caregivers. An in-ear sensor has advantages in user acceptance (it resembles a hearing aid) and continuous use, but poses unique challenges since only head accelerations (no gyroscope) are available. Due to the lack of a dedicated ear-worn fall dataset, we use the public SisFall database (waist-mounted IMU) for development. SisFall comprises 19 types of ADLs and 15 types of falls performed by young and older adults [1] and its creators report that even a simple threshold-based classifier can achieve up to 96% fall-detection accuracy on it [1]. We will build on these data with three design concepts (a simple threshold detector, a single ML classifier, and a hybrid threshold+ML approach) and compare their performance.

II. Related Work

Numerous studies compare threshold-based fall detectors to machine-learning methods. Threshold detectors use fixed cut-offs on acceleration magnitude or orientation changes. These are simple and fast, and some implementations (e.g. using an accelerometer sum magnitude threshold) have reported high accuracy in controlled tests (sensitivities around 92–100% and specificities above 95%) [5], [3]. However, these algorithms tend to fail on diverse real-world data: small movements or fall-like activities (e.g. quick sitting) can trigger false alarms, and variations between users can violate fixed thresholds. By contrast, machine learning (ML) classifiers can learn complex patterns from data. For example, Aziz et al. (2017) evaluated five ML methods (e.g. SVM, decision tree) versus five literature thresholds using waist accelerometer data. They found that all ML models outperformed threshold rules, with a support-vector machine giving the best balance of sensitivity and specificity [6]. Other work also reports very high ML-based performance: Yu et al. used a Hidden Markov Model on raw accelerations to achieve 99.2% sensitivity (fall) and 99.0% specificity [5]. Likewise, Liu et al. extracted a rich time-domain feature set from a waist accelerometer and reached 97.6% accuracy with an SVM [7]. ; Chelli et al. reported 99.1% accuracy for fall-versus-ADL classification using an ensemble tree classifier on multiple features [7].

Recent reviews conclude that ML and even deep-learning models often exceed the accuracy of simple thresholds [6], [5]. Nevertheless, ML classifiers require more computation and power. Hybrid approaches have therefore been proposed. For example, Li et al. (2009) designed a fall detector using both accelerometers and gyroscopes: by combining linear acceleration and angular velocity, they reduced both false positives and negatives compared to accelerometer-only methods [4]. In a similar spirit, a 2024 study introduces a two-tier hybrid algorithm: it first applies simple acceleration/gyro thresholds to identify candidate fall windows, then applies a trained ML model to classify those windows as "fall" or "ADL." This reduced computation (since the ML model runs only on likely events) while maintaining high accuracy [5]. These hybrid schemes align with our third concept.

Sensor placement is another important factor. Most studies use the waist or chest, which are close to the body's center of mass and yield the strongest signals. A systematic analysis found the waist sensor gave the highest fall-detection sensitivity (99.96%), whereas peripheral placements (e.g. wrist) performed worse [8]. Very few works examine ear-worn sensors for fall detection. However, activity-tracking research shows ear-mounted accelerometers can still recognize common motions: for example, classifying 9 ADL categories yielded ≈84% accuracy with an ear sensor (vs. 90% from a waist sensor) [9]. This suggests ear-level sensing is feasible, albeit somewhat lower-performing than torso mounts. (In fact, modern hearing aids now include accelerometers for context-adaptive features, with fall-detection mentioned as a potential application [9].) In summary, while ear placement is less studied, existing evidence indicates it can capture useful motion features, though we should expect some accuracy loss compared to chest/waist sensors [8], [9].

III. System Design and Data Preprocessing

Our goal is to detect falls from the earpiece's accelerometer in real time. We consider three designs:

- **Threshold-based detector**: Compute the magnitude of acceleration $|\mathbf{a}| = \sqrt{a_x^2 + a_y^2 + a_z^2}$ continuously. If $|\mathbf{a}|$ exceeds a preset threshold (e.g., $\sim$1.4–1.5 g, as in prior work [10]) within a short time window, signal a fall. This simple method has low computational requirements but is prone to false alarms when sudden non-fall movements occur. In practice, we would tune the threshold to balance missed falls versus false alarms, as in [5] who achieved $\sim$92.6% sensitivity using thresholding.
- **Continuous ML classifier**: Segment the incoming acceleration into fixed-size time windows (e.g., 2–3 seconds), extract features from each window, and feed them to a trained classifier (e.g., SVM, Random Forest, neural network) that predicts "fall" or "no fall." This mirrors the approach of many studies: raw accelerometer data are first filtered (e.g., low-pass at $\approx$10 Hz, since human motion is mostly $<$10 Hz [5]), then for each window we compute time-domain statistics (mean, variance, peak, skewness, etc.) and perhaps frequency-domain metrics [7], [10]. The classifier is trained on labeled fall vs. ADL (Activities of Daily Living) windows. Previous works report that such ML models can achieve high accuracy (often $>$95%) on balanced test sets [6], [7].
- **Hybrid threshold + ML**: A two-tier strategy that combines the above. First, apply a quick threshold check: only when the acceleration magnitude in a window briefly exceeds a secondary (lower) threshold do we invoke the ML classifier. Windows below this level are immediately labeled "no fall." This reduces computational workload and false positives. A similar approach was used in recent research [5]: candidate fall "events" are detected by thresholding the acceleration magnitude (or signal magnitude vector, SMV), and then refined by an ML model. In our case, once the earpiece's SMV exceeds the chosen cut-off, we extract features from a surrounding 2-second window and feed them to the ML model to decide *fall* vs. *benign activity* [5]. If the threshold is never crossed, we presume ADL (no fall) and need not run the classifier at all.

All approaches use features derived from accelerometer windows. Common practice is to filter the raw data (we will apply a 4th-order Butterworth low-pass filter at $\sim$10 Hz [5]) and compute the signal magnitude vector (SMV):

$$\text{SMV} = \sqrt{a_x^2 + a_y^2 + a_z^2} \tag{1}$$

Falls produce a sharp SMV peak when the body impacts the ground [5]. We plan to align windows to this peak (e.g., taking 1 s before to 1 s after) as candidate fall samples [5]. For ADL windows, we will slide a fixed-length window through the data (as done in [5]) on segments where no fall peaks occur. From each window we will extract features (mean, max, variance of each axis, SMV peak, energy, etc.) as is standard in the literature [7], [10].

We will train and evaluate classifiers using the *SisFall* dataset. While *SisFall*'s IMU is waist-mounted and includes both accelerometer and gyroscope data [1], our earpiece provides only accelerometry. Consequently, we will use only the acceleration channels from *SisFall*, ignoring the gyroscopic data. We note this as a potential limitation: many high-accuracy fall detection systems rely on multimodal sensor data [4], so using only accelerometer data (particularly from a head-worn position rather than waist-mounted) may reduce detection performance.

Our preprocessing pipeline (filtering, window segmentation) follows established practices from these benchmark datasets [1], [7]. Specifically:

- Apply 4th-order Butterworth low-pass filtering ($\sim$10 Hz cutoff)
- Compute SMV ($\sqrt{a_x^2 + a_y^2 + a_z^2}$) for each sample
- Segment data into event-aligned windows (1 s pre-peak to 1 s post-peak for falls)
- Extract time-domain features (mean, variance, peak magnitude, etc.)

A key component in wearable fall detection systems is the design of input features used for classification. Numerous studies have shown that hand-crafted statistical and spectral features derived from accelerometer signals can significantly improve machine learning model performance [11], [7].

In most high-performing systems, data are first segmented into short windows (typically 1–3 seconds), and then multiple types of features are computed from these windows. These features fall into three main categories:

- **Time-domain features**: mean, variance, standard deviation, RMS, range, signal magnitude area (SMA), skewness, kurtosis, and zero-crossing rate [11], [12]
- **Frequency-domain features**: FFT coefficients, spectral entropy, power in different frequency bands, and dominant frequency peaks [7], [13]
- **Cross-axis or composite features**: inter-axis correlations (Pearson or Kendall), jerk magnitude (first derivative of acceleration), and resultant acceleration vectors [11]

The study by [11], which used an ear-worn accelerometer, extracted a rich set of 64 features per segment—including absolute means, RMS, min/max, interquartile ranges, skewness, kurtosis, signal energy, and spectral entropy—achieving $\sim$95% accuracy across six ADLs using a Support Vector Machine (SVM). Notably, they found that simple time-domain features like variance, range, and minimum of the vertical axis were among the most discriminative for fall and ADL classification from an ear-worn position.

Similarly, [12] used a waist-mounted sensor and reported that nine features per axis—including median, percentiles, and entropy—were sufficient to train models like SVM and Random Forests with accuracy in the range of 93–96%, especially when generalizing to unseen fall scenarios.

[7] extracted 72 features, including autocorrelation peaks and amplitudes, band powers (e.g., 0–5 Hz, 5–10 Hz, 10–20 Hz), and reported an F1-score of $\sim$98.4% for binary fall detection using SVMs, and $\sim$88% for multi-class fall and ADL detection. Importantly, they also showed that feature

selection could reduce the input set to just 13 features without significantly harming performance, indicating that redundancy among features can be minimized through dimensionality reduction.

[13] further supported the value of hybrid feature types, combining RMS and FFT-based spectral features with auto-correlation and wavelet packet decomposition (WPD) to boost classification performance on smartphone-based ADL data to ∼98% when using SVMs. In contrast, using time-domain features alone led to significantly lower accuracy (∼74%).

Across these studies, a consistent pattern emerges: the most effective ML-based fall detectors combine multiple feature types to represent motion dynamics in both time and frequency domains. Simple, interpretable features—such as mean, RMS, range, entropy, and inter-axis correlation—are not only effective but also computationally efficient, making them ideal for low-power wearable implementations. These findings validate the design choices in our system, where we extracted 33 carefully selected time- and frequency-domain features, such as spectral entropy, RMS, jerk, velocity energy, and correlation coefficients, tailored to an accelerometer-only earpiece device.

Furthermore, the cited studies reinforce the point that high performance is possible even without gyroscopes, provided the features are informative enough and segmentation aligns with key signal events (e.g., fall peaks). This supports the practicality of our accelerometer-only setup using the LIS2DW12 sensor, despite its limitations in orientation sensing [11], [7].

IV. Methodology

This section presents the design process for our earpiece-based fall detection system. The methodology includes the conceptual model, data preparation, feature extraction, model design, and evaluation setup.

A. Conceptual Design Models

To explore reliable fall detection, we defined three fundamentally different detection models based on system requirements:

1) **Threshold-based detection**: This model detects falls when a fixed threshold of acceleration magnitude is exceeded. Although simple and fast, it was prone to false alarms—especially when high-movement ADLs (e.g., jogging or stairs) created large peaks.
2) **Full ML-based detection**: A machine learning classifier continuously monitors all signals and classifies them into five categories: walking, jogging, stairs, sitting, and falls. This model relies on extracted features from raw sensor data to perform multi-class classification at every time window.
3) **Hybrid model (Threshold + ML)**: This approach combines a simple threshold-based trigger with a lightweight ML model. When the acceleration exceeds the threshold, the ADL classifier is activated to distinguish a fall from a high-intensity ADL. In other times, the ADL classifier runs at a lower frequency to label ongoing daily activities for context awareness.

B. Dataset and Preprocessing

Due to the lack of publicly available data for the commercial earpiece (LIS2DW12, 200 Hz, 3-axis accelerometer), we used the publicly available SisFall dataset, which contains labeled ADL and fall recordings at the same sampling rate (200 Hz). Our final model is trained using SisFall and evaluated both on test splits and unseen earpiece data collected in-house.

Activity Selection

From the SisFall dataset, we selected 5 activity types:

- **ADLs**: walking, jogging, going up/down stairs, sitting
- **Falls**: F01–F15, covering various fall scenarios

To increase similarity between SisFall and our earpiece signals:

- We swapped X and Y axes to align sensor orientation.
- We converted raw values to acceleration in g, considering ADXL345's configuration (±16g, 13-bit resolution).
- Only accelerometer channels were used, to reflect our hardware constraints.

Signal Segmentation:

In real-world fall detection, recognizing subtle differences between normal daily activities and falls is highly dependent on the quality of extracted signal segments. For this reason, we manually and algorithmically separated each activity from the raw recording sessions, instead of treating long continuous recordings as input. This approach was chosen based on three motivations:

- **Improved Label Accuracy**: In public datasets like SisFall, activities are often recorded in sessions (e.g., walk for 30 seconds, then sit, then fall). Segmenting individual activities ensures the label of each window precisely corresponds to a single activity.
- **Reducing Overlap/Noise**: Continuous sessions can contain transitions between activities or idle periods. Segmenting ensures that only the relevant portion is used in training, which reduces confusion during classification.
- **Matching Real-Time Detection Constraints**: Our system will classify activities in short windows (4 seconds),

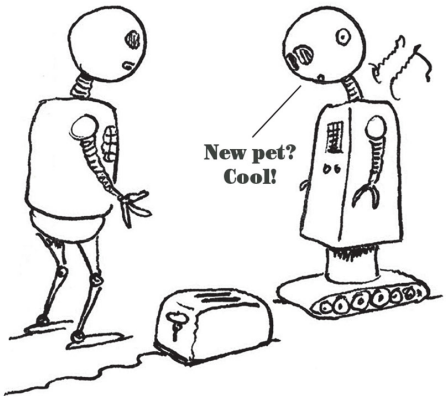

Fig. 1. Insert Conceptual Model Diagram here — Figure 1 [20].

so preparing the training data in the same structure improves model generalizability.

Segmentation Method

- Window Size: We used fixed-length windows of 800 samples (4 seconds), balancing between capturing enough motion and maintaining quick detection latency.
- Fall Activities: For fall segments, we applied a peak detection algorithm:
  - Calculated the standard deviation magnitude across gyroscope axes using a sliding window.
  - Identified the highest activity zones (peaks).
  - Centered a 4-second window around the peak to capture the fall motion.
- ADLs: For low-intensity or non-violent activities like sitting or walking:
  - Used `idle_remover()` to eliminate flat or static parts (e.g., sitting idle).
  - Applied `split_and_add()` or `split_and_center()` to generate clean windows from high-movement sections only.
- Length Fixing: For segments shorter than 800, zero-padding or duplication was applied to preserve shape uniformity required by classifiers.

[Insert Figure: Gyro STD peak detection + window overlay on fall signal]

C. Feature Extraction

We implemented two feature extraction functions, returning 14 and 33 features per window respectively. These features were selected based on previous research and domain-specific heuristics to capture key movement characteristics.

14-Feature Set

These features emphasize temporal movement patterns and were designed for lightweight classification:

- **C1–C2**: Root Mean Square (RMS) of total and horizontal acceleration
- **C3**: Peak-to-peak RMS magnitude

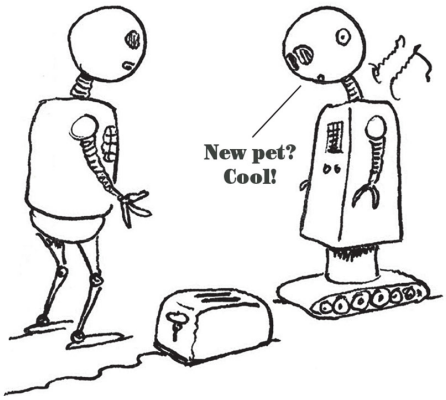

Fig. 2. [Insert Example Plot — Figure 2: Peak Detection on Fall Signal] [20].

- **C4–C5**: Orientation indicators (tilt angle, trunk angle)
- **C6–C8**: Orientation change and jerk
- **C9–C14**: Composite velocity energy, Signal Magnitude Area (SMA), and cumulative displacement

Used in lightweight models and tested for generalization to earpiece data.

33-Feature Set This extended set combines time-domain, frequency-domain, and correlation-based features:

- **C1–C20**: Same as above + per-axis mean, std, skewness, kurtosis
- **C21–C30**: Frequency features (entropy, dominant frequency, band power 0–5 Hz)
- **C31–C33**: Cross-axis correlations (X–Y, Y–Z, X–Z)

These features were particularly useful for distinguishing subtle variations between falls and fast ADLs, as seen in prior work.

[Insert Table: Full 33-feature definitions with equations or descriptions]

D. Model Design and Training

We compared a diverse set of classifiers from different categories to explore tradeoffs between accuracy, generalization, and inference complexity:

TABLE I
OVERVIEW OF CLASSIFIER MODELS USED

Model Type	Algorithms Used	Notes
Tree-based	Random Forest, Decision Tree, Extra Trees, Gradient Boosting	High accuracy and feature ranking ability
Linear models	Logistic Regression, Linear Discriminant Analysis (LDA)	Fast, interpretable
Distance-based	k-Nearest Neighbors (k=3)	Sensitive to input scale
Probabilistic	Naive Bayes	Assumes independence
Kernel-based	SVM (RBF/Polynomial Kernel)	Effective with nonlinear boundaries

All models were implemented using Scikit-learn, with hyperparameters kept at default or tuned minimally. We used:

- **Stratified K-Fold Cross-Validation**: 5-fold CV to ensure balanced class distribution in each fold
- **Train/test split**: 80% training, 20% testing with stratification
- **Grid Search**: For hyperparameter tuning, especially for SVM and tree-based models
- **Evaluation Metrics**: Accuracy, precision, recall, F1-score

We excluded models requiring GPU acceleration (e.g., deep learning) due to the limited power budget of the earpiece hardware.

Models were trained and tested on three classification tasks:

1) **Binary Classification**: Fall vs. ADL
2) **Multi-class ADL**: Walking, Jogging, Stairs, Sitting
3) **Multi-class ADL+Fall**: Walking, Jogging, Stairs, Sitting, Fall

Each model was also evaluated on unseen earpiece recordings, segmented similarly, to assess generalizability.

[Insert Table — Table 2: Overview of Models and Tasks]

[Insert Confusion Matrix Samples — Figures 3–5]

E. Limitations and Practical Considerations

The major limitation in this methodology is the difference in sensor placement between the SisFall dataset and our ear-worn device. While we aligned axes and matched sampling rates, real-world signals from the ear show slightly different dynamics. As observed, models with higher feature counts performed better on test data but degraded on unseen data, highlighting the sensitivity of ML models to sensor context and orientation.

Additionally, the absence of gyroscope data on the final device constrained signal quality during peak detection and orientation estimation.

V. Formatting Requirements

You are free to write the report in LaTeX or Word (a template is available for LaTeX in Overleaf[1]). The report should adhere to the IEEE conference paper format, which is a two-column layout. The official IEEE template can be downloaded from the IEEE page[2]. Following the formatting instructions in the template, all formatting shall be done automagically. Note, this guideline is also written in the IEEE template style.

VI. Content Structure

The report is structured to provide a comprehensive overview of the design and implementation process. Key sections include an introduction that outlines the context and problem being addressed, a literature review, a conceptual model (requirements, overview of the proposed solution, research question or goal), a detailed description of the design of the process to answer the research question (or to achieve the goal) and how to obtain and validate the results, results and discussion. Finally, the report concludes with a summary of findings and potential areas for improvement. To ensure reproducibility and transparency, code and other relevant information must be available and linked (for example, on GitHub). The suggested structure of your report is as follows. This structure is not set into stone, hence, you are free to adjust it to your needs.

- **Title and Author Information**: Begin with the title of your project and your name(s) as the author(s). Include your university affiliation and email.
- **Abstract**: A brief summary of your report, not exceeding 200 words. May be the last part to be written. The abstract should not contain an introduction to the topic.
- **Keywords**: List 3–5 keywords that best describe your robot design and programming.
- **Introduction, Motivation**: Describe the background and importance of the topic, including relevant statistics and a specific description of the problem that you are addressing and the current situation. Context of company/research institute is important. List the requirements and possible ethical and environmental considerations. It may also be advisable to place a teaser figure (like Fig. 3) right at the beginning of the paper, so that the reader becomes familiar with the topic more quickly.
- **Literature Review**: Here you describe a few related works and compare state-of-the-art methods, including quantitative comparisons for key methods. Opportunities for improvement of methods or application are identified. Note that all literature used is referenced and listed at the end of the paper.
- **Conceptual Model**: Give an overview of the proposed solutions, list the corresponding requirements, and compare variants. Include a graphical overview of solution presented, like Figure 4. State the goal or the (S.M.A.R.T.) Research Question, and make sure to prioritize the sub-goals or sub-questions.
- **Design**: Step-by-step approach to answer the Research Question or achieve the goal. Describe each step with evaluation/validation methods. Ethical/Environmental aspects are taken into consideration. Include a section on how to validate results. It is always a good idea to include figures to help with the understanding.
- **Results**: This section provides experimental data with a qualitative analysis, supported by illustrations. Results should be presented in the same order as the design. Figures must be cited and explained in the text, and must have captions with a concise description. Graphs must contain proper scale and units.
- **Discussion**: Interpret and analyze your results, compare them to existing literature, explain unexpected results, acknowledged limitations of the setup.
- **Conclusion**: Summarize the main findings, discuss the significance of the study, answer the Research question / goals, evaluate the fulfillment of the requirements, and present suggestions for future research.
- **Acknowledgments**: help from other persons, like fund-

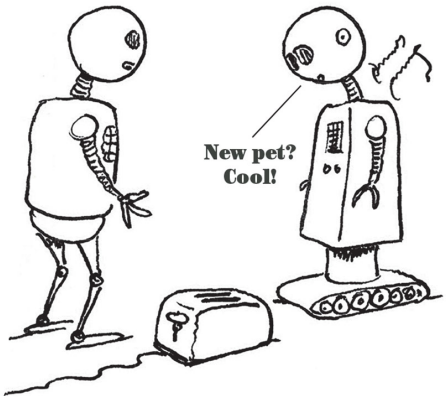

Fig. 3. Comic by Scott Masear, adapted from [20].

[1]https://www.overleaf.com

[2]https://www.ieee.org/conferences/publishing/templates.html

TABLE II
EXAMPLE OF A TABLE

Feature	ESP32	RP2040
Manufacturer	Espressif Systems	Raspberry Pi Foundation
Core Architecture	Xtensa Dual-Core (32-bit)	ARM Cortex-M0+ Dual-Core
Clock Speed	Up to 240 MHz	Up to 133 MHz
RAM	520 KB SRAM	264 KB SRAM
Flash Memory	Up to 16 MB (external SPI flash)	External flash via QSPI (no internal flash)
GPIO Pins	Up to 36 GPIO	Up to 30 GPIO
Connectivity	Wi-Fi, Bluetooth	None (requires external modules for connectivity)
Peripherals	UART, SPI, I2C, ADC, DAC, PWM, etc.	UART, SPI, I2C, ADC, PWM, PIO, etc.
Power Consumption	Moderate (Wi-Fi increases power use)	Low (optimized for low power)
Development Ecosystem	Arduino IDE, ESP-IDF, MicroPython	Arduino IDE, MicroPython, C/C++ SDK
Price Range	$3–$10	$1–$4

ing, proof-reading, and ideas or results adopted from other sources should be acknowledged.
- **Contributions**: For each author, itemize the contributions to the project, e.g. which sections have been written, which parts implemented, etc.
- **References**: List all sources cited (and only the ones cited) in the text.

VII. Submission Guidelines

- **File format**: Submit your report as a **PDF file**.
- **Naming convention**: Name the file as: `YourGroupnumber_Report.pdf`.
- **Length**: maximum 6 pages

VIII. Usage of AI Tools

You are permitted to use AI tools to assist in the preparation of your project report, but their use is strictly limited to reformatting and rephrasing text. You can also use such tools to assist in coding (like CoPilot). These tools should not be employed for generating content or ideas, as the report must reflect your original work and understanding. If you choose to use an AI tool, you are required to declare which model(s) was(were) used and explicitly specify the sections where it was applied. This ensures transparency and adherence to academic integrity. Failure to disclose the use of AI tools may result in penalties, so be diligent and honest in documenting their use.

The report must not contain any plagiarism, not even as modified text.

IX. Style Guide

A. Bibliography

LaTeX offers its own environment for creating bibliographies. Make use of it! As literature sources, only peer-reviewed publications, books, and technical reports are appropriate. These are papers with authors, title and publication date, for journals with volume/number, for conferences optionally with conference date and location. You may also name the publisher and page numbers, optionally a URL. Examples for proper references are [15], [16], [17], [18].

B. Web Links

Web links, blogs, etc. do not count as literature, since the internet is subject to continuous change and its contents have not gone through a professional reviewing process. Online-sources can either be listed separately, or be added to the text as footnotes.

C. Professional Style

The style of writing for scientific papers is focused and factual, without digress (no ”marketing”, no jokes, no ”sloppy” language). The ”I” form (and also the imperative, except for exercises) should be avoided. You can use the ”we” form if you are writing in a way that includes the reader (also when you are the sole author, e.g. when writing a thesis). As a rule, almost everything is written in present tense. If at all, the future tense is used in the last section. The past tense can be used when dating related work.

D. Figures, Tables, Equations

It is important that figures and tables (flow objects created by LaTeX placed at the top of the page) are provided with numbers and captions containing a meaningful description.

Each of these objects should be referenced (at least once) within the text where it is relevant. The same applies to literature: every reference should appear at least once, including one or two sentences dedicated to it. When working with BibTeX, only those references that are used appear, at all. This way one can re-use the same literature base for multiple papers.

Equations are automatically numbered in round brackets by LaTeX, and this number is used when you make references to the equation using its label (just like Figures). The numbering of equations is very helpful for sharing when writing between multiple authors and reviewers. Equations are considered part of the text, so they must contain punctuation marks when applicable. Besides, all its variables need to be explained in the text.

An example is the calculation of the current through the LED connected to pin 1 of the ESP32 in the circuit illustrated by Figure 4. The current I_{LED} is calculated as

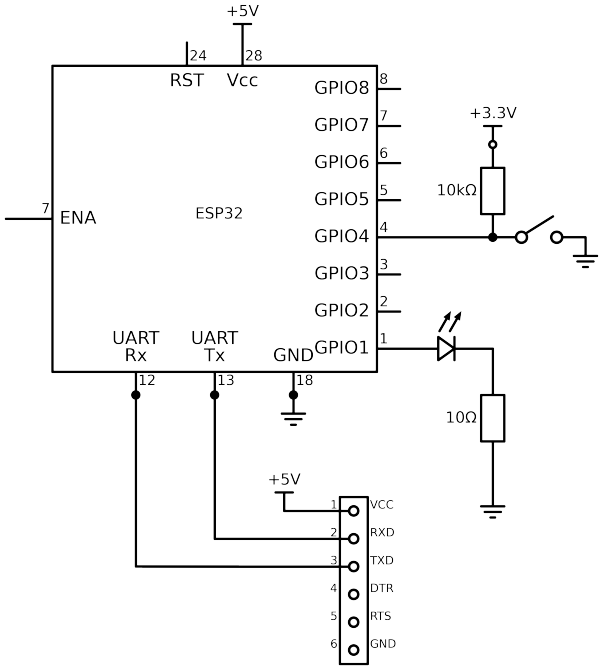

Fig. 4. Example Schematic generated with Schemdraw [21].

$$I_{LED} = \frac{V_R}{R}, \tag{2}$$

where V_R is the voltage across the 10Ω resistor connected in series with the LED, and R is its resistance value. Equation (2) is known as Ohm's Law.

E. Acknowledgments

Acknowledgments: Those whom advice has been taken from (other than the authors) and in case results are shared with other developers, these people should also be named and their contributions should be acknowledged. Acknowledgments do not diminish the value of your own work. On the contrary: They show that you have also exchanged ideas with other experts about the solution approaches and the technologies used.

X. Before Submitting

Before submitting your report, read it again considering the following points:

- Are title and abstract appropriate and do they motivate to read the paper?
- Research: Are proper references to related work contained?
- Technical correctness: Are the descriptions complete and free of errors? Is the presentation in scientific notation with proper units?
- Reproducibility: Is it possible to reproduce the results from the information given?
- Contribution: Does the paper contain authentic examples and evaluations?
- Readability: Is the paper well written and easy to understand?
- Length: Is the length justified by the technical contribution?
- Overall Impression: Does it comply with formal criteria (language, references to figures and literature, structure, completeness, etc.)? Is it easy to read and understand?
- Check for typos and further issues.

XI. Conclusions

By following these guidelines, you can ensure that your report is well-written and adheres to scientific standards. This approach will help the readers to clearly understand your ideas, implementations and contributions. Consequently, this will enhance the quality of your submission and potentially result in a higher grade.

XII. Acknowledgments

This text is based on the original "Report Submission Guidelines for BIP Project" (Jan. 2025), by S.F. Peik (HSB), F. Martins (Hanze), J. Lima (IPB), Â. Ferreira (IPB), and M. Hering-Bertram (HSB). Content in Section VI is based on the proposal made by Gijs van der Schot.

References

[1] Sucerquia, A., López, J. D., & Vargas-Bonilla, J. F. (2017, January). *SisFall: A Fall and Movement Dataset*, Sensors, 17(1), 198. https://www.mdpi.com/1424-8220/17/1/198

[2] Guilbeault-Sauvé, A., De Kelper, B., & Voix, J. (2021, March). *Man Down Situation Detection Using an in-Ear Inertial Platform*, Sensors, 21(5), 1730. https://www.mdpi.com/1424-8220/21/5/1730

[3] Aziz, O., & Robinovitch, S. N. (2011). *An analysis of the accuracy of wearable sensors for classifying the causes of falls in humans*, IEEE Transactions on Neural Systems and Rehabilitation Engineering, 19(6), 670-676. https://doi.org/10.1109/TNSRE.2011.2162250

[4] Li, Q., Stankovic, J. A., Hanson, M. A., Barth, A. T., Lach, J., & Zhou, G. (2009). *Accurate, fast fall detection using gyroscopes and accelerometer-derived posture information*, In Proceedings of the 2009 Sixth International Workshop on Wearable and Implantable Body Sensor Networks (BSN) (pp. 138-143). https://doi.org/10.1109/BSN.2009.46

[5] Wang, Y., Sarvari, P. A., & Khadraoui, D. (2024). *Fusion of Machine Learning and Threshold-Based Approaches for Fall Detection in Healthcare Using Inertial Sensors*, In Proceedings of the 17th International Joint Conference on Biomedical Engineering Systems and Technologies (BIOSTEC 2024), vol. 1, pp. 573-582. https://doi.org/10.5220/0012250500003657

[6] Aziz, O., Musngi, M., Park, E. J., Mori, G., & Robinovitch, S. N. (2016). *A comparison of accuracy of fall detection algorithms (threshold-based vs. machine learning) using waist-mounted tri-axial accelerometer signals from a comprehensive set of falls and non-fall trials*, *Medical & Biological Engineering & Computing*, 55 (1), 45–55. https://doi.org/10.1007/s11517-016-1504-y

[7] Althobaiti, T., Katsigiannis, S., & Ramzan, N. (2020). *Triaxial Accelerometer-Based Falls and Activities of Daily Life Detection Using Machine Learning*, Sensors, 20(13), 3777. https://doi.org/10.3390/s20133777

[8] Özdemir, A. T. (2016). *An Analysis on Sensor Locations of the Human Body for Wearable Fall Detection Devices: Principles and Practice*, Sensors, 16(8), 1161. https://doi.org/10.3390/s16081161''

[9] Skoglund, M. A., Balzi, G., Jensen, E. L., Bhuiyan, T. A., & Rotger-Griful, S. (2021). *Activity Tracking Using Ear-Level Accelerometers*, Frontiers in Digital Health, 3, Article 724714. https://doi.org/10.3389/fdgth.2021.724714

[10] Palmerini, L., Klenk, J., Becker, C., & Chiari, L. (2020). *Accelerometer-Based Fall Detection Using Machine Learning: Training and Testing on Real-World Falls*, Sensors, 20(22), 6479. https://doi.org/10.3390/s20226479

[11] Boborzi, L., Decker, J., Rezaei, R., Schniepp, R., & Wuehr, M. (2024). *Human Activity Recognition in a Free-Living Environment Using an Ear-Worn Motion Sensor*, Sensors, 24(9), 2665. https://doi.org/10.3390/s24092665

[12] Alizadeh, J., Bogdan, M., Classen, J., & Fricke, C. (2021). *Support Vector Machine Classifiers Show High Generalizability in Automatic Fall Detection in Older Adults*, Sensors, 21(21), 7166. https://doi.org/10.3390/s21217166

[13] Bennasar, M., Price, B. A., Gooch, D., Bandara, A. K., & Nuseibeh, B. (2022). *Significant Features for Human Activity Recognition Using Tri-Axial Accelerometers*, Sensors, 22(19), 7482. https://doi.org/10.3390/s22197482

[14] Gjoreski, M., Lustrek, M., & Gams, M. (2011). *Accelerometer placement for posture recognition and fall detection*, In Intelligent Environments (IE), 2011 7th International Conference on (pp. 47–54). https://doi.org/10.1109/IE.2011.16

[15] Nira Dyn, David Levin and John A. Gregory, *A 4-point interpolatory subdivision scheme for curve design*, Computer Aided Geometric Design, vol. 4, no. 4, Elsevier, 1987, pp. 257-268.

[16] Gerald Farin, *Curves and Surfaces for CAGD. A practical guide.* 5. Auflage, Academic Press, San Diego 2002, ISBN 1-55860-737-4

[17] Kenneth Levenberg *A Method for the Solution of Certain Problems in Least Squares*, Quarterly of Applied Mathematics, vol. 2, no. 2, American Mathematical Society (AMS), 1944, pp. 164-168. https://www.ams.org/journals/qam/1944-02-02/home.html

[18] Donald W. Marquardt, *An Algorithm for Least-Squares Estimation of Nonlinear Parameters*, SIAM Journal on Applied Mathematics, vol. 11, no. 2, Society for Industrial and Applied Mathematics (SIAM), 1963, pp. 431-441. https://epubs.siam.org/doi/abs/10.1137/0111030

[19] Justin Zobel, Writing for Computer Science, 3rd edition, Springer, 2014.

[20] Robot jokes, https://jokes.scoutlife.org/topics/robot-jokes/, accessed 6.1.2025

[21] Collin J. Delker, Schemdraw, https://pypi.org/project/schemdraw/